



Day 6 Notes

Webcam + Canvas Basics – My Try-On Track Begins

★ Today = my role split from my friend's for real. She's building the store (product pages, cart). I started the TRY-ON feature, beginning with the webcam.

1 *Why I'm Starting Here*

The try-on feature needs: webcam video → detect body → draw clothes on top, live. Today was just step one: get the webcam working and draw something on top of it.

Used a plain HTML file (no React yet) in a new practice folder, since this is pure learning before touching the real app.

2 *Showing the Webcam on a Page*

```
<video id="webcam" width="640" height="480" autoplay></video>

<script>
const video = document.getElementById('webcam')

navigator.mediaDevices.getUserMedia({ video: true })
  .then((stream) => { video.srcObject = stream })
  .catch((err) => console.error('Webcam error:', err))
</script>
```

- `<video>` → an HTML element that can show live video
- `getUserMedia({ video: true })` → asks the browser for permission to use the camera
- `video.srcObject = stream` → connects the live camera feed to the video element

Browsers block camera access on plain `file://` pages, so I had to run a tiny local server: `npx serve`.

3 *Troubleshooting: Permission Didn't Pop Up*

First try, no permission prompt appeared. Fixed by checking:

- Browser console (F12) for errors
- The camera icon in the address bar, in case it was already blocked
- Whether another app (like another tab) was already using the webcam

Good reminder: camera APIs can be finicky, and checking the browser console is always step one when debugging.

4 *Drawing on Top: The Canvas Layer*

A <canvas> is a transparent drawing layer. Stacked exactly on top of the video using CSS, it looks like one combined image:

```
<div style="position: relative; width: 640px; height: 480px;">
  <video ... style="position: absolute; top: 0; left: 0;"></video>
  <canvas id="overlay" ... style="position: absolute; top: 0; left:
0;"></canvas>
</div>
```

5 *The Draw Loop*

```
function drawLoop() {
  ctx.clearRect(0, 0, canvas.width, canvas.height)

  ctx.beginPath()
  ctx.arc(320, 200, 20, 0, 2 * Math.PI)
  ctx.fillStyle = 'red'
  ctx.fill()

  requestAnimationFrame(drawLoop)
}
drawLoop()
```

- `clearRect(...)` → wipes the previous drawing so it doesn't smear
- `arc(...) + fill()` → draws a filled circle (a stand-in for a future "detected point")
- `requestAnimationFrame(drawLoop)` → calls itself again next frame, about 60 times per second

Result: a red dot floating on top of my live webcam feed, redrawn constantly. Looks simple, but this exact pattern (video + synced canvas drawing) is how the real clothing overlay will work later — just with a clothing image instead of a dot, and real body coordinates instead of a fixed position.



Big takeaway:

`getUserMedia` = how to access the webcam in the browser. video + canvas stacked together = the foundation for drawing anything on top of live video. The draw loop (`clearRect` + redraw + `requestAnimationFrame`) is the exact mechanism my clothing overlay will use. Next step (Day 7): replace the fixed red dot with a REAL detected point, using MediaPipe Pose.